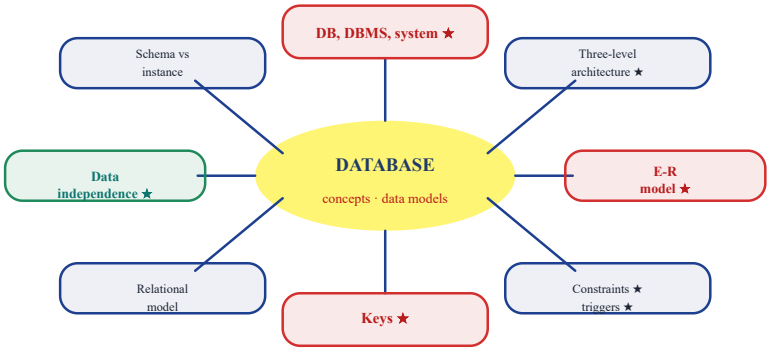
Database Concepts & Data Models

DB · DBMS · three levels · E-R · relational · keys · constraints · triggers



↑ the whole topic: red = high-yield, green = the idea the architecture exists for

Syllabus coverage

The Database Systems syllabus reads: "Concepts of database systems; Data Models: **Entity-Relationship model, Relational model**; Query Languages: Relational algebra, SQL; **Constraints and triggers**; Functional dependencies and normalization; File organization and data storage; Indexing; Query optimization; Transaction management; Recovery; Concurrency control; Access control and security; Semi-structured database; Object oriented databases." **This lesson takes the first three clauses only.**

High-yield list

① DB system = **DB + DBMS + apps + users**
② Schema = blueprint · instance = data now
③ External → Conceptual → Internal
④ Independence: **logical**, **physical**
⑤ Chen: rect = entity, ellipse = attribute, diamond = relationship, underline = key
⑥ Weak entity = **double rectangle**
⑦ 1:N ⇒ "one" side's PK as FK in the "many" side
⑧ M:N ⇒ **needs its own junction table**
⑨ FK ⇒ **referential**, PK ⇒ **entity** integrity
⑩ **DROP** kills structure, DELETE/TRUNCATE do not

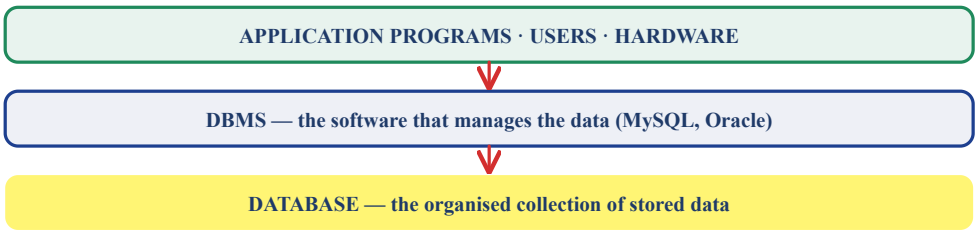
Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

Contents

✓ Database, DBMS, database system … p. 2
✓ Three levels; schema vs instance … p. 3
✓ Data independence … p. 4
✓ E-R model, Chen's notation … p. 5
✓ Cardinality 1:1, 1:N, M:N … p. 6
✓ The relational model … p. 7
✓ Keys … p. 8
✓ Constraints & integrity … p. 9
✓ Triggers; E-R vs relational … p. 10
✓ Night-before sheet … p. 11
✓ Practice MCQs + written … p. 12

1 Database, DBMS and Database System

Definition: A **database** is an organised collection of related data. A **DBMS** is the software that stores, retrieves and protects it. The **database system** is the whole assembly: **database + DBMS + application programs + users + hardware**. Every "which one is X?" question turns on which of those five a name denotes.



DB = data, DBMS = software

three words, one stack

Term	What it actually is	Examples
Database	the data itself, in an organised persistent form	students, courses, exams of a university
DBMS	software sitting between the application and the data	MySQL, Oracle, PostgreSQL, SQL Server; MongoDB is *NoSQL*
Database system	**all five parts together**	what a university actually installs and runs
File system	the OS's files: no rules, no shared access	what the database replaced

Why databases replaced files: Four failures, four answers: **redundancy** (a fact stored in several files) → one schema; **inconsistency** (copies disagree) → **constraints**; **poor sharing** → views and a query language; **program–data dependence** (change the file layout, rewrite every program) → the DBMS owns the storage.

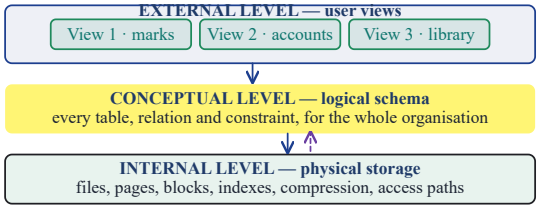
Memory trick: **"Data, software, everything."** Is the DBMS the database? **No** — the DBMS is the manager. Is the database the DBMS? Also no. Only **database system** means the whole assembly.

Exam trap: A **database is not a table**, and a **file system is not a database**: files enforce no schema, no transactions, no concurrency. "Which one enforces integrity?" → the **DBMS**.

Worked example ✍️ an ATM: The **database** is the account table. The **DBMS** decrypts the PIN, locks the row so nobody withdraws at the same instant, checks the balance and writes the new one. The **application** is the screen menu; the **database system** is bank software + core server + card machine + network + customer.

2 The Three-Level Architecture

Definition: The **three-level (three-schema) architecture** of **ANSI/SPARC** splits the database into an **external** level (what each user sees), a **conceptual** level (the logical structure of the whole organisation's data) and an **internal** level (how it is physically stored). Mappings join them — and the mappings are what make **data independence** possible.



each arrow is a mapping that can be changed

Level	Question it answers	Who sees it
External	"What does *this* user see?"	one user or application; many views
Conceptual	"What data exists, and what does it mean?"	the whole organisation's logical view
Internal	"Where and how is it stored?"	the DBMS alone

Worked example ✍️ schema vs instance, one table: The **schema** is `Student(id, name, dept)` — the blueprint, changed rarely, kept in the **data dictionary**. The **instance** is `(101,"Alice","CSE")`, `(102,"Bob","ECE")` — the values *right now*. **Schema changes slowly; instance changes constantly.**

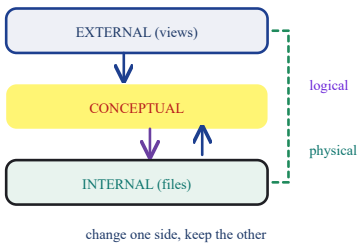
Point	Schema	Instance
Also called	logical design, metadata	snapshot, current state
Contains	names, columns, types, constraints	the actual rows stored
Changed	rarely, by the designer (DDL)	constantly, by every insert (DML)
Stored in	the data dictionary / catalogue	the data files

Memory trick: **"Outside, Concept, Inside."** *E* = **E**very **u**ser, *C* = **C**ommunity view of all data, *I* = **I**nternal plumbing. Read the stack **top down**: from the user's words to the bytes on disk.

Exam trap: The **external level is not one view** — there are normally several. The **internal level is private to the DBMS**; no program is written against it. The **conceptual level is logical**: no file names, no block sizes.

3 Why Three Levels? Data Independence

Definition: **Data independence** is the ability to change one level of the schema without changing the level above. There are exactly **two** kinds; a paper offering "three levels of independence" is testing this. **Logical** is conceptual → external; **physical** is internal → conceptual.



two arrows, two kinds

The two kinds, precisely: **Logical data independence** — change the **conceptual schema** (add a column, split a table, add a table) and the **external views and programs stay as they were**. Harder, because the meaning of data changes.
Physical data independence — change the **internal schema** (add an index, split a file, reorganise pages, change compression) and the **conceptual schema and programs stay as they were**. Easier, because the meaning does not change.
Both are delivered by the two mappings: **external/conceptual** and **conceptual/internal**.

	Logical data independence	Physical data independence
Between	conceptual level and external level	internal level and conceptual level
You change	the **logical** schema: add `phone`, split a table	the **storage**: add a B+ tree index, reorganise pages
Unaffected	external views and application programs	conceptual schema and application programs
Holds because	the external/conceptual **mapping** is rewritten	the conceptual/internal **mapping** is rewritten
Typical example	adding a column without touching any query	dropping an index changes speed, not meaning
Difficulty	**harder** — the meaning changes	**easier** — only the layout changes

Memory trick: **"Logical is on top, physical is underneath."** Read the arrow upwards in each pair: logical independence changes the level *above*, physical the level *below*.

Exam trap: There are **two** kinds, not three. Adding an index is **physical**; adding a column is **logical**; reformatting a file without touching the schema is the classic **physical** example.

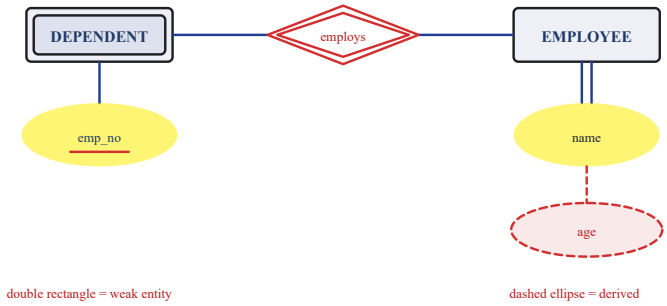
Worked example ✍ name the change:
"The admin rebuilds the table with a hash index on *dept*." → **physical**: the queries still mean the same. "A column *nationality* is added to Student." → **logical**: the conceptual schema changed.

IT link: This is the reason a dump from one engine can be reloaded into another, and the reason `ALTER TABLE … ADD COLUMN` on a big table is operationally cheaper than rewriting every program that reads that table. A **view** is the external level made real: it stores a query, not a copy of the data.

4 The Entity-Relationship Model in Chen's Notation

Definition: The **E-R model** (Peter Chen, **1976**) is a high-level **conceptual** model used to design a database before any table is built. It has three primitive components — **entity**, **attribute**, **relationship** — and every one has a fixed symbol.

> Chen, 1976



weak entity, identifying relationship, key, derived attribute

Component	Symbol	Example
Strong entity	rectangle	Employee
Weak entity	**double rectangle**	Dependent
Attribute	ellipse	name, id
Key attribute	**underlined**	id
Derived	dashed ellipse	age
Multivalued	double ellipse	phones
Relationship	diamond	Enrolls
Identifying	double diamond	owns
Cardinality	1, N, M on arms	1:N, M:N
Total particip.	double line	every emp has a dept

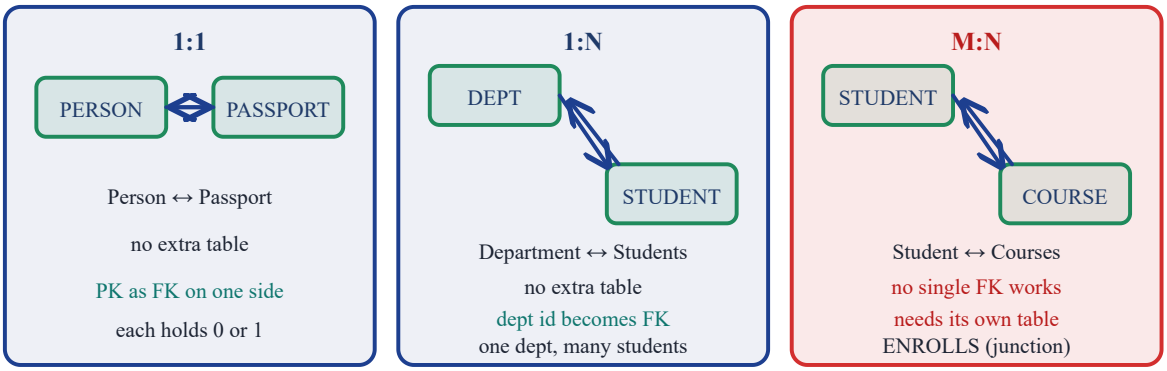
Exam trap: A **weak entity set is a double rectangle** — not a diamond (relationship), not an ellipse (attribute), not a dashed ellipse (derived attribute). Its identifying relationship is a **double diamond**. A weak entity has **no key of its own**: it is identified by the *owner's key + its own partial key*.

Memory trick: "**Rectangle entity, ellipse attribute, diamond relationship, underlined = key.**" Then one letter per exception: **double** rectangle = weak, **dashed** ellipse = derived, **double** ellipse = multivalued, **double** diamond = identifying. A **composite** attribute is an ellipse with sub-ellipses (*address* = house, street, city).

Worked example ✍️ reading an E-R diagram: "Every **DEPARTMENT** rectangle has one **name** underlined and a dashed ellipse **no_of_staff**. A **double rectangle ASSIGNMENT** joins PROJECT to it." — *name* is a key attribute; *no_of_staff* is **derived**; **ASSIGNMENT** is a **weak entity**, meaningless until you know which project and which employee it belongs to.

5 Cardinality: 1:1, 1:N and M:N

Definition: **Cardinality** states how many instances of one entity may be associated with one instance of the other. There are only three shapes — **1:1**, **1:N**, **M:N** — and the shape decides **how many relations the design needs**.



a 1:N relationship costs no extra table; an M:N relationship always costs one

the shape alone tells you the number of tables

Cardinality	Meaning	Example	Mapping to relations
1:1	one A, at most one B	Person ↔ Passport	no new table: put the "one" side's **PK as an FK** in the other table
1:N	one A, many B	Department ↔ Students	no new table: **the "one" side's PK as an FK in the "many" side**
M:N	many A, many B	Student ↔ Courses	**its own relation** whose FKs point to both sides
1:N with data	the relationship itself carries *since*, *grade*	Student ↔ Enrols	the junction table becomes necessary even for 1:N

Exam trap: For **1:N the foreign key goes on the N side**; on the "one" side it would need several ids in one cell, which breaks 1NF. An **M:N cannot be written with any single foreign key**: a student has many courses *and* a course has many students, so neither table can hold the other's list. **Only M:N forces a new relation.**

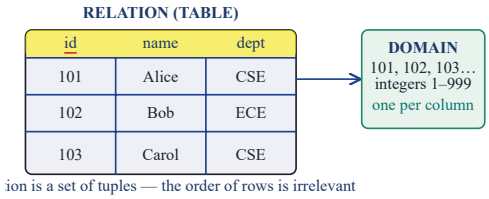
Worked example ✍️ why M:N needs a table: Try to put the link in `Student`: the column `courses` would hold `{CSE101, MAT201}` — a **multi-valued cell**, so one student gives one value per column, exactly what 1NF forbids. Put it in `Course` and the same problem appears. Therefore **ENROLLS(student_id, course_code)**, with an FK to each side; the pair is usually the composite key too.

Choosing the side for a 1:1: Put the foreign key on the side that may be **missing**: the employee table gets `passport_no`, so an employee without a passport stores NULL, whereas putting it on `Passport` would demand a passport row for every employee. For **1:N the side is not a choice** — the key always goes on the N side.

6 The Relational Model

Definition: The **relational model** (E. F. Codd, 1970) represents data as **relations** — mathematical sets of tuples over named attributes. A relation is shown as a table: tuples are rows, attributes are columns. Two properties matter in every answer: a relation has **no duplicate tuples** and **no order**.

> Codd, 1970



rows in, rows out; columns carry domains

Formal term	Everyday name	Meaning
Relation	table	a set of tuples with the same attributes
Tuple	row / record	one fact: `(101,"Alice","CSE")`
Attribute	column / field	a named property: *name*, *dept*
Domain	allowed value set	integers 1–999 for *id*
Degree	arity, #columns	Student has degree **3**
Cardinality	#rows	Student holds 3 tuples
Relation schema	table definition	`Student(id, name, dept)`

Formal properties: Cells hold **atomic** values only · tuples are **unique** · tuple and attribute **order does not matter** · every tuple has the same attributes in the same domains.

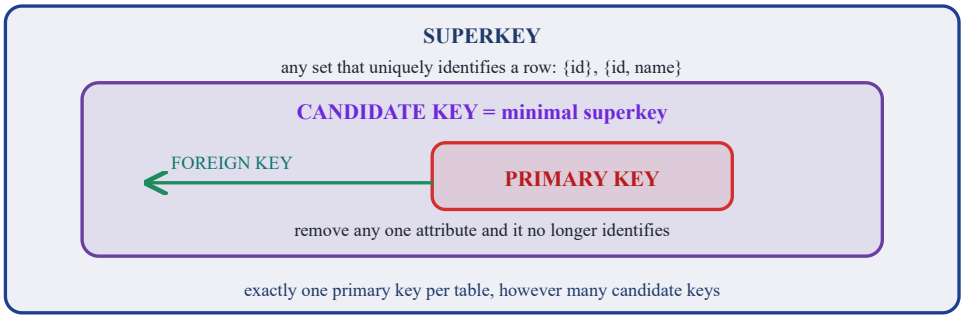
Memory trick: "**Table–Row–Column–Set.**" "Is degree rows or columns?" → **columns**. "Is cardinality rows or columns?" → **rows**. Degree and cardinality are the pair examiners swap.

Exam trap: A table **may be sorted on screen**, but the relation is a set, so **row order is not part of the data**. **Degree does not change** when rows are added. Two identical rows are **not** allowed — that is what the primary key prevents.

Worked example ✍️ describe one table formally: `Student(id, name, dept)` holding `(101,"Alice","CSE")`, `(102,"Bob","ECE")`, `(103,"Carol","CSE")`. Relation name **Student**; degree **3** (three attributes); cardinality **3** (three tuples); domain of *id* = integers 1–999; *id* underlined, so it is the key attribute. Adding a fourth student raises cardinality to 4 and leaves degree at 3.

7 Keys: Super, Candidate, Primary, Foreign, Composite

Definition: A **superkey** is any set of attributes that uniquely identifies a tuple. A **candidate key** is a superkey that is **minimal** — remove one attribute and it no longer identifies. The **primary key** is the single candidate key chosen as the official identifier. A **foreign key** points at another relation's candidate key. A **composite key** is built from two or more attributes.



composite key = a key built from two or more attributes

every key is a superkey; not every superkey is a key

Key	What it is	Rule	Example
Superkey	identifies a row	may be larger	{id, name}
Candidate key	**minimal** superkey	nothing extra	{id}
Primary key	the chosen key	**one per table; never NULL**	PRIMARY KEY (id)
Foreign key	points at another key	**referential** integrity	`dept_id REFERENCES Dept(id)`
Composite key	2 or more attributes	a PK may be composite	(StudentID, CourseID)
Alternate key	a key not chosen	still UNIQUE	{passport_no}

Memory trick: "**Every candidate key is a superkey; every primary key is a candidate key; a foreign key is somebody else's primary key.**" The nesting runs one way: superkey ⊇ candidate key ⊇ primary key.

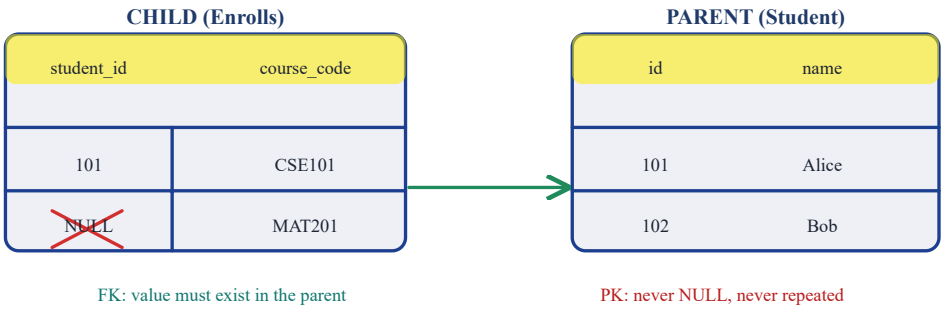
Exam trap: A table has **exactly one** primary key but many candidate keys and many superkeys. A primary key may be **composite** — it need not be one column. A foreign key column **may** hold NULL; only its non-NULL values must match.

Worked example ✍ classify the keys of one table: `Person(pid, passport_no, phone)`. {pid} unique → superkey; remove *pid* and it fails → minimal → **candidate key**; chosen → **primary key**. {passport_no} is minimal too → **alternate key**. {pid, phone} is unique but **not** a candidate key, since *phone* can be dropped.

8 Constraints and the Two Integrity Rules

Definition: A **constraint** is a rule declared on a column or a table that restricts what may be stored, so the data stays correct however it is written. SQL gives six constraint forms, and they exist to protect two named properties: **entity integrity** and **referential integrity**.

two integrity rules



the FK arrow may not point at nothing

Constraint	What it forbids	SQL
NOT NULL	a NULL in the column	`name VARCHAR(50) NOT NULL`
UNIQUE	two rows sharing a value (NULL excepted)	`email VARCHAR(100) UNIQUE`
PRIMARY KEY	NULLs and duplicates — implies the two above	`id INT PRIMARY KEY`
FOREIGN KEY	a value matching no key of the referenced table	`dept_id INT REFERENCES Dept(id)`
CHECK	a value failing a stated condition	`CHECK (Age >= 18)`
DEFAULT	nothing — supplies a value when none is given	`status VARCHAR(20) DEFAULT 'active'`

The two integrity rules constraints protect: **Entity integrity — the primary key is never NULL** and is unique in every row. A relation must be addressable: a row with no identity cannot be pointed at. Enforced by **PRIMARY KEY**. **Referential integrity — every non-NULL foreign key value must match an existing key value in the table it references**. No orphans. Enforced by **FOREIGN KEY**; on deleting the parent you must choose `CASCADE`, `SET NULL`, `SET DEFAULT` or `RESTRICT`.

```
CREATE TABLE Enrolls ( student_id  INT      NOT NULL REFERENCES Student(id),
                       course_code CHAR(7)  NOT NULL REFERENCES Course(code),
                       grade       CHAR(2)  CHECK (grade IN ('A','B','C','F')),
                       enrolled_on DATE     DEFAULT CURRENT_DATE );
```

Exam traps ✍️ two of them: **1.** A foreign key exists to enforce **referential integrity** — not entity integrity, encryption or faster sorting; entity integrity is the primary key's job. A FK column **may** hold NULL. **2. DROP** removes the table's **structure and its data**; **DELETE** removes rows (optionally by `WHERE`) and **TRUNCATE** removes all rows, but **both keep the structure**; **ALTER** changes the structure and keeps the data.

9 Triggers, and E-R Model vs Relational Model

Definition: A **trigger** is stored logic the DBMS executes **automatically when an event happens** on a table — before, after, or instead of an `INSERT`, `UPDATE` or `DELETE`. The application never calls a trigger; the *engine* fires it, which is why it cannot be bypassed by any client.

Timing	Fires
BEFORE	before the row is written; may change `NEW`
AFTER	after the row is written; `OLD` and `NEW` both available
FOR EACH ROW	once per affected row, not per statement
INSTEAD OF	on a **view**, to make it updatable

Exam trap: A trigger is **not** a stored procedure: nobody executes it by name, it fires on an **event**. It is not a substitute for a constraint — a `CHECK` is cheaper and clearer. **Foreign-key enforcement is done by the constraint, not by a trigger.**

```
CREATE TRIGGER trg_student_count
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
  UPDATE Department
  SET StudentCount = StudentCount + 1
  WHERE DeptID = NEW.DeptID;
END;
```

Worked example ✍️ three uses: **1. Maintain a count:** the trigger above adds 1 to that department's head-count after every insert. **2. Audit:** after any update of `salary`, copy the old and new value into `Salary_Log`. **3. Validate:** before inserting, reject a booking dated in the past — though a `CHECK` is preferred.

Aspect	E-R model (conceptual)	Relational model
Stage	**design** — before any table exists	**implementation** — the tables themselves
Data as	entities, attributes, relationships	relations, tuples, attributes
Abstraction	high — logical, implementation-free	low — close to storage
Users	designers and analysts	programmers and DBAs
Maps to the	**conceptual level** of the three-level architecture	**external and internal levels** of the implementation

Where the rest of this chapter lives: **Relational algebra and SQL** (selection σ, projection π, joins; SELECT, WHERE, GROUP BY, HAVING) → the query-languages lesson. **Functional dependencies, 1NF–BCNF, normalisation** → the normalisation lesson. **File organisation, indexes, B+ trees, hash tables, optimisation** → the indexing lesson. **ACID, transactions, concurrency, recovery** → the transactions lesson.

Night-Before-the-Exam Sheet

Fact sheet:

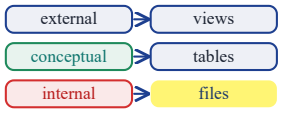
DB system = DB + DBMS + apps + users + hardware	DB = the data; DBMS = the managing software
Schema = blueprint; instance = data now	External → Conceptual → Internal (ANSI/SPARC)
External = views · Conceptual = logical · Internal = physical	Independence: logical and physical — two kinds
Chen: entity = rectangle	Attribute = ellipse · key = underlined
Relationship = diamond · identifying = double diamond	Weak entity = **double rectangle** · derived = dashed
1:1 and 1:N need no extra table	M:N always needs its own junction relation
Degree = #columns · cardinality = #rows	Relation = set of tuples; no duplicates, no order
Superkey ⊇ candidate key ⊇ primary key	One primary key per table; never NULL
FK → referential · PK → entity integrity	DROP kills structure; DELETE/TRUNCATE do not

Top 12 one-liners for MCQs

1. Database system ≠ database ≠ DBMS
2. Schema = blueprint, instance = the data now
3. ANSI/SPARC: external, conceptual, internal
4. Data independence has exactly **two** kinds
5. Adding an index is **physical** independence
6. Chen's entity is a rectangle, key an underlined ellipse
7. A weak entity set is a **double rectangle**
8. For 1:N the FK goes on the **N side**
9. Only M:N forces a junction table
10. Degree = columns, cardinality = rows
11. FK enforces **referential** integrity
12. **DROP** removes structure; TRUNCATE does not

Top traps: Weak entity is not a diamond or an ellipse · one primary key but many candidate keys · a primary key may be composite · a FK column may be NULL · two kinds of independence, not three · degree and cardinality swapped · DROP vs DELETE vs TRUNCATE.

All tricks together: Rectangle entity, ellipse attribute, diamond relationship, underlined key · double rectangle weak, dashed ellipse derived · one dept, many students ⇒ FK on the many side · M:N needs its own table · superkey ⊇ candidate ⊇ primary · E-ntity integrity = no NULL PK, R-eferential = FK must land.



three levels, one meaning

Worked example ✍️ one design, three relations: "Department 1:N Students, Students M:N Courses." → `Student` gets a **dept_id** FK (the N side); a separate `Enrolls(student_id, course_code)` appears, because M:N cannot use one FK. "How many relations?" → **3**.

✓ Practice MCQs (exam pattern, 10 marks)

1. A database system has:
(a) the DBMS alone (b) the database alone
(c) DB + DBMS + apps + users (d) the data files alone

2. Schema and instance are:
(a) schema = structure (b) schema = data now
(c) both in the data files (d) instance = blueprint

3. Adding a B+ tree index without code change is:
(a) logical independence (b) physical independence
(c) the external level (d) a schema change

4. Chen's weak entity set is drawn as:
(a) a diamond (b) an ellipse
(c) a dashed ellipse (d) a double rectangle

5. 1:N DEPARTMENT : EMPLOYEE maps as:
(a) DEPT's PK as FK in EMPLOYEE (b) EMP's PK as FK in DEPT
(c) merge both tables (d) a third table

6. Student ↔ Course is M:N because:
(a) many attributes (b) no single FK works
(c) links cannot span two tables (d) it needs two indexes

7. A foreign key constraint mainly enforces:
(a) entity integrity (b) data encryption
(c) referential integrity (d) faster sorting

8. Which of these is a composite key?
(a) id (b) (StudentID, CourseID)
(c) name (d) the whole row

9. A trigger differs from a procedure:
(a) needs Java (b) cannot modify data
(c) is called by the app (d) is run by the DBMS

10. Which drops structure and data?
(a) DELETE (b) TRUNCATE
(c) ALTER (d) DROP

Answer key → 1-c · 2-a · 3-b · 4-d · 5-a · 6-b · 7-c · 8-b · 9-d · 10-d

Written question: Define a database system and distinguish schema from instance. Explain the three-level architecture and the two kinds of data independence. Draw Chen's E-R notation including a weak entity set, show how 1:1, 1:N and M:N map to relations, and state what constraints and triggers are for. 10 marks

1. **Definition (1)** — database = the data; DBMS = the managing software; database system = database + DBMS + applications + users + hardware. **Schema** = the blueprint, kept in the data dictionary; **instance** = the rows now. 1
2. **Three levels (2)** — **external** (user views, one per application), **conceptual** (logical structure of all data: tables, relations, constraints), **internal** (files, pages, indexes). ANSI/SPARC. 2
3. **Data independence (2)** — **logical**: change the conceptual schema; the external views and programs are unaffected. **physical**: change the internal schema (index, page layout); the conceptual schema and programs are unaffected. 2
4. **E-R notation (2)** — entity = rectangle (weak = **double rectangle**), attribute = ellipse (key **underlined**, derived **dashed**), relationship = **diamond** (identifying = double), cardinality 1/N/M. 2
5. **Mapping (2)** — **1:1** and **1:N**: no new relation; put the "one" side's primary key as a foreign key in the other / in the "many" side. **M:N**: no single FK works, so create a junction relation. 2
6. **Constraints and triggers (1)** — declared rules (NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT) protecting **entity integrity** (PK never NULL) and **referential integrity** (FK matches an existing key); triggers are stored logic run automatically on an event. 1

Before the exam ✓

- ✓ Can separate database, DBMS, database system
- ✓ Can give the three levels in order
- ✓ Can name the two kinds of independence
- ✓ Can draw all Chen symbols from memory
- ✓ Can state why M:N needs a table
- ✓ Can list the six constraints, DROP vs TRUNCATE